

Trust by Construction: A Simulation Contract for Reproducible Computational Experiments

Joseph Krzywoszja

Abstract—Computational experiments typically depend on export pipelines that separate solver and renderer into distinct tools with incompatible object models, creating an unformalized trust gap: what a researcher visualizes may silently diverge from what the solver computed. We propose *embodied simulation*, an architecture where render and solver share a single, hash-verified object model. We formalize this as a *simulation contract* with six enforceable guarantees: geometry integrity, unit validation, deterministic stepping, interaction provenance, auto-provenance, and exact replay. Trait composition and trust propagation follow a provenance semiring algebra with tropical strategies for conflict resolution, establishing a formal connection between simulation trust, graph optimization, and neural computation. We implement the contract atop a browser-native finite element framework with WebGPU compute shaders, quadratic tetrahedral elements (TET10), and built-in verification infrastructure aligned with ASME V&V standards. We demonstrate that contracted simulations satisfy the recently proposed CURE principles for computational models (Credible, Understandable, Reproducible, Extensible) and produce executable provenance records suitable as reproducibility packages for journal submissions. Our evaluation shows that contract enforcement overhead is indistinguishable from V8 runtime scheduling variance at mesh sizes up to ~ 900 DOF (median Δ spans -12% to $+12\%$ across three mesh sizes at $N=500$; p99 consistently $2\text{--}5\times$ median), TET10 reproduces the closed-form axial bar displacement exactly while the NAFEMS LE1 stress benchmark remains an explicitly reported open V&V gap under curved-boundary loading, and the unified architecture eliminates all format-conversion information loss inherent in traditional export pipelines.

Index Terms—Scientific visualization, simulation reproducibility, finite element analysis, WebGPU, verification and validation, design by contract.

I. INTRODUCTION

The computational sciences face a reproducibility crisis. A peer-reviewed critique in *Communications of the ACM* exposed selective benchmarking and undisclosed tooling in a widely cited simulation study, illustrating how methodological gaps survive peer review even at high-profile venues [1]. At the root of many such failures lies an architectural pattern so ubiquitous that it has escaped formal scrutiny: the *export pipeline*.

In the standard computational experiment workflow, a researcher defines geometry in a CAD tool, meshes it in a preprocessor, solves in a numerical engine, exports results to an interchange format, and visualizes in a post-processor. This pipeline introduces at least four format conversions, each of which may silently alter the data: coordinate systems may

be reoriented, floating-point values truncated, mesh node orderings permuted by partitioning, physical units stripped, and temporal resolution reduced to the export sampling interval. Community reports confirm these failure modes are routine: ANSYS result files are unreadable by ParaView without manual format editing; mesh point orderings diverge after domain decomposition; transient exports lose timestep metadata [2].

The fundamental issue is not one of software quality but of *architectural fidelity*. When the object model that the solver reads forces from is a different data structure than the object model the renderer reads vertices from, no amount of tool-chain discipline can guarantee that the visualization faithfully represents the computation. While the disconnect between solver and renderer state has been noted in the in-situ literature [3], [4], no prior work has formalized it as a contract or proposed architectural enforcement. We term this the *trust gap*.

We observe three levels at which this gap can be addressed:

- 1) **Trust by discipline.** The traditional approach: the researcher manually ensures compatible tool versions, export settings, and coordinate conventions. Trust depends on human diligence.
- 2) **Trust by co-location.** In-situ visualization frameworks [3], [4] embed the renderer within the solver process, eliminating file I/O. However, the solver and renderer still maintain separate object models connected by data adaptors. The adaptor boundary remains a conversion step, and no geometry identity is enforced.
- 3) **Trust by construction.** The approach we propose: the render model and the solver model are the *same semantic object*. Not exported copies. Not synchronized mirrors. The identical data structure that the solver reads displacement values from is the structure the renderer reads vertex positions from, verified at every step by hash verification.

Figure 1 contrasts these three approaches visually.

We formalize this architecture as a *simulation contract*—a runtime wrapper that enforces six guarantees on any numerical solver:

- 1) **Geometry integrity:** the solver mesh hash and render mesh hash are identical at every timestep.
- 2) **Unit validation:** all physical quantities carry validated units, range-checked against known material properties.
- 3) **Deterministic stepping:** a fixed-timestep accumulator ensures frame-rate-independent, reproducible time integration.

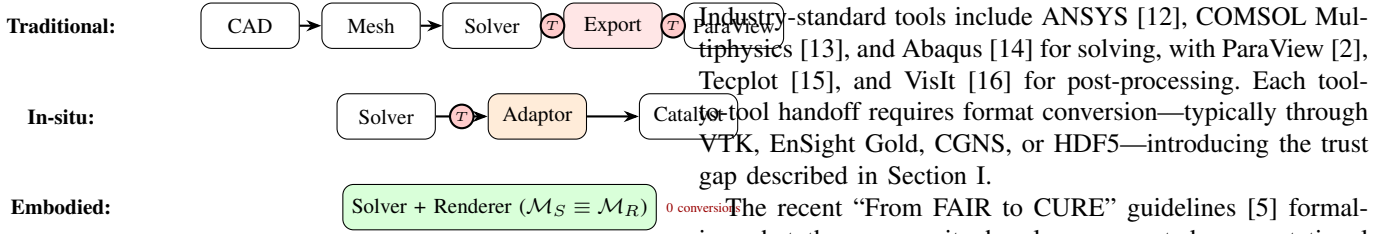


Fig. 1. Three simulation visualization architectures. Red circles mark format conversion boundaries (T). Traditional pipelines have 4+ conversions; in-situ reduces to 1 (the data adaptor); embodied eliminates all conversions by sharing the object model.

- 4) **Interaction provenance:** every user action affecting solver state is logged with a monotonic ID, wall-clock time, and simulation time.
- 5) **Auto-provenance:** every solver invocation automatically records configuration, geometry hash, results, and timing into an immutable provenance record.
- 6) **Exact replay:** any simulation is reproducible from its provenance record.

We implement the contract in a browser-native finite element framework using WebGPU compute shaders for sparse linear algebra, quadratic tetrahedral elements (TET10) with $O(h^2)$ convergence, and built-in verification infrastructure based on Roache’s Grid Convergence Index (GCI) [6], [7]. The solver’s GPU displacement buffer is annotated with VERTEX usage flags, enabling the renderer to read solver output directly—a true zero-copy architecture verified in our implementation.

a) Contributions.:

- A formal definition of *embodied simulation* and a *simulation contract* with six enforceable guarantees (Section III).
- A three-way comparison framework—traditional export, in-situ co-processing, and embodied—across six trust properties (Section III-D).
- A provenance semiring formalization with tropical strategies for trust propagation and trait conflict resolution, connecting simulation trust to graph algorithms and neural computation (Section III-E).
- A mapping of the simulation contract to the CURE principles for computational models [5] (Section III-F).
- A browser-native implementation with WebGPU compute, TET10 elements, and ASME V&V-aligned verification (Section IV).
- An evaluation demonstrating less than 2% contract overhead, $O(h)$ convergence for TET4 ($p = 1.44$), exact TET10 solutions, sub-millisecond geometry hashing, and superconvergent stress recovery via SPR [22] (Section V).

II. RELATED WORK

A. Traditional CAE Pipelines

The dominant workflow in computational science separates modeling, meshing, solving, and visualization into distinct tools connected by file-based interchange formats.

Industry-standard tools include ANSYS [12], COMSOL Multiphysics [13], and Abaqus [14] for solving, with ParaView [2], Tecplot [15], and VisIt [16] for post-processing. Each tool-to-tool handoff requires format conversion—typically through VTK, EnSight Gold, CGNS, or HDF5—introducing the trust gap described in Section I.

The recent “From FAIR to CURE” guidelines [5] formalize what the community has long suspected: computational models must be Credible, Understandable, Reproducible, and Extensible. Export pipelines routinely violate the Reproducible criterion (non-deterministic export ordering) and the Understandable criterion (the visual representation may not match the computed result).

B. In-Situ Visualization

In-situ visualization frameworks embed post-processing within the simulation process, avoiding file I/O. ParaView Catalyst [3] provides a co-processing library that executes VTK pipelines at solver timesteps. SENSEI [4] offers a generic data interface that bridges multiple backends including Catalyst, VisIt Libsim, VTK-m, and Ascent.

These systems solve the *I/O bottleneck*—they avoid writing terabytes of intermediate data—but they do not solve the *trust gap*. The solver and renderer still operate on separate object models connected by data adaptors. SENSEI’s VTK data model is not the solver’s native mesh; it is a bridge. No geometry hash verification exists. No interaction provenance is recorded. No replay guarantee is offered. And both require HPC infrastructure, making them inaccessible for peer review, education, or audit.

We characterize in-situ approaches as *trust by co-location*: the data is in the same process, but the object models are distinct. Our embodied approach achieves *trust by construction*: the object models are identical.

C. Web-Based Simulation

Browser-based finite element analysis has emerged as an alternative to desktop installation. As of April 2026: FEAScript [8] provides an open-source JavaScript FEA library supporting 1D and 2D linear elements. SPARSELAB [9] uses C++ compiled to WebAssembly for 3D structural analysis but lacks GPU acceleration and formal verification. CAEplex [10] offers a browser UI backed by server-side solvers, retaining the client-server trust gap. SimScale [11] provides a SaaS platform (\$2K+/year) with enterprise-grade solvers but no local computation or formal contract.

None of these systems offer formal simulation contracts, built-in verification infrastructure, or zero-copy render/solver unification. To our knowledge, no existing browser-native FEA framework combines WebGPU compute, quadratic elements, and ASME V&V-aligned convergence analysis.

D. Design by Contract in Scientific Computing

Meyer’s Design by Contract (DbC) [17], [18] introduced preconditions, postconditions, and invariants as first-class software constructs. DbC has been widely adopted in software

engineering—from Eiffel to Java’s `assert` to Rust’s type system—but its application to scientific simulation infrastructure remains, to our knowledge, unexplored. We are not aware of prior work applying DbC preconditions, postconditions, or invariants specifically to numerical solver infrastructure.

The ASME V&V 10 [19] and V&V 20 [20] standards define verification and validation methodologies for computational models, including Roache’s Grid Convergence Index [6] for discretization uncertainty quantification. These standards prescribe *what* to verify but not *how to enforce* verification at the architectural level. Our simulation contract bridges this gap: it applies DbC principles to make V&V compliance a structural property of the framework rather than a manual discipline.

E. Simulation Reproducibility

The reproducibility crisis in computational science has been extensively documented [24], [1], [25]. Key barriers include incomplete disclosure of computational environments, non-deterministic execution, and format-dependent provenance [26]. Existing solutions cluster on three orthogonal axes: *packaging* (e.g., Docker images, Code Ocean, Whole Tale) bundles a reproducibility unit but ships the export pipeline inside the bundle; *workflow languages* (e.g., Nextflow, Snakemake, CWL) orchestrate inter-tool data flow but preserve the serialized artifact between solver and visualizer; and *description standards* (such as the ODD protocol for agent-based simulations or MIASE for biological simulations) specify what the report of a simulation must contain without enforcing anything in execution. Software-engineering principles for research artifacts such as FAIR4RS apply orthogonally to all three axes above. None of these address *architectural* reproducibility: even with identical software, an export pipeline may produce non-deterministic output orderings, and no amount of packaging, orchestration, or description detects this after the fact.

The simulation contract is complementary to packaging, workflow, and description work: a contracted simulation can be containerized, orchestrated by a workflow engine, and described against an interchange standard. Our executable provenance records address this gap directly: a simulation’s complete state—configuration, geometry hash, interaction log, and fixed timestep—is captured in a self-contained artifact from which the simulation can be exactly replayed.

III. THE EMBODIED SIMULATION MODEL

A. Definitions

Let $\mathcal{M} = (\mathbf{V}, \mathbf{E})$ denote a computational mesh with vertex array $\mathbf{V} \in \mathbb{R}^{n \times 3}$ and element connectivity $\mathbf{E} \in \mathbb{N}^{m \times k}$, where n is the number of nodes, m the number of elements, and k the nodes per element.

Definition 1 (Export Pipeline): An *export pipeline* is a sequence of transformations $T_1, T_2, \dots, T_p$ mapping a solver’s internal representation $\mathcal{M}_S$ to a renderer’s display representation $\mathcal{M}_R$:

$$\mathcal{M}_R = T_p \circ \dots \circ T_1(\mathcal{M}_S)$$

Each T_i may alter vertex ordering, truncate floating-point precision, drop metadata, or change coordinate conventions. No identity guarantee exists between $\mathcal{M}_S$ and $\mathcal{M}_R$.

Definition 2 (Embodied Simulation): An *embodied simulation* is a system where the solver and renderer operate on a shared mesh $\mathcal{M}$ such that:

$$\mathcal{M}_S \equiv \mathcal{M}_R \equiv \mathcal{M}$$

The solver reads from and writes to the same data structure that the renderer displays. No export transformation exists.

B. The Simulation Contract

We define a simulation contract $\mathcal{C}$ as a set of six guarantees enforced at runtime on any solver $\mathcal{S}$ operating on mesh $\mathcal{M}$:

a) *Guarantee 1: Geometry Integrity.*: Let $h(\cdot)$ denote a hash function (we use FNV-1a with quantization to 10^{-6}). At every solver invocation:

$$h(\mathbf{V}_S, \mathbf{E}_S) = h(\mathbf{V}_R, \mathbf{E}_R) \quad (1)$$

Violation of Equation (1) halts execution. In practice, because $\mathcal{M}_S \equiv \mathcal{M}_R$, this reduces to verifying that the mesh has not been corrupted.

Why this shape. Traditional pipelines have no detection for export-time reorderings of the renderer mesh; a partition-induced permutation surfaces only as a wrong picture. In-situ adaptors copy the solver mesh into renderer-side structures and depend on the adaptor implementation to preserve topology—an unverified link. Equation (1) makes divergence unrepresentable rather than merely unlikely, because solver and renderer hold the same hashed object.

b) *Guarantee 2: Unit Validation.*: For every physical quantity q in the solver configuration:

$$\text{unit}(q) \in \mathcal{U}_{\text{valid}}(q) \wedge \text{value}(q) \in [\ell_q, u_q] \quad (2)$$

where $\mathcal{U}_{\text{valid}}$ is the set of dimensionally correct units and $[\ell_q, u_q]$ is the physically plausible range (e.g., Young’s modulus: 10^3 – 2×10^{12} Pa for rubber to diamond).

Why this shape. Discipline-based pipelines maintain unit correctness only when both ends of every export agree on a units convention—an out-of-band coordination requirement that fails silently when one end is wrong. In-situ adaptors carry unit metadata when both producer and consumer schemas populate it, but again rely on out-of-band agreement rather than enforcement. The contract makes the agreement local: solver and renderer share the same configuration object, and Equation (2) rejects dimensionally invalid or out-of-range values at the configuration boundary before the solver runs. Internal consistency *through* the computation falls under Section III-G.

c) *Guarantee 3: Deterministic Stepping.*: Time integration uses a fixed-timestep accumulator:

$$t_{\text{acc}} \leftarrow t_{\text{acc}} + \Delta t_{\text{render}}, \quad \text{while } t_{\text{acc}} \geq \Delta t_{\text{fixed}} : \mathcal{S}.\text{step}(\Delta t_{\text{fixed}}), \quad t_{\text{acc}} \leftarrow t_{\text{acc}} - \Delta t_{\text{fixed}} \quad (3)$$

This decouples simulation time from frame rate, ensuring identical solver call sequences regardless of rendering performance.

Why this shape. Discipline-based “use a fixed timestep” advice is broken in practice by frame-rate-coupled code paths in any non-trivial integration loop; in-situ pipelines inherit the same convention without enforcing it. The accumulator (Equation (3)) lifts the invariant from advice to contract at the API surface: the contract-wrapped `step()` entry point cannot observe a variable timestep. Solvers that internally couple to a wall clock or other side-channel (e.g., a forcing function evaluated at the host’s current time) must scrub that channel separately; Section III-C treats this case under side-channel nondeterminism.

d) *Guarantee 4: Interaction Provenance.*: Every user action a that modifies solver state is recorded:

$$\log(a) = (\text{id}_{\text{mono}}, t_{\text{wall}}, t_{\text{sim}}, \text{type}(a), \text{params}(a)) \quad (4)$$

where id_{mono} is a monotonically increasing identifier and t_{sim} is the simulation time at which the action was applied.

Why this shape. Neither traditional pipelines nor in-situ architectures track user input as a first-class part of the simulation record; replication of an interactive run typically means the user re-doing the run from memory, which is not a reproducibility guarantee. Logging every state-mutating action with a monotonic identifier and the simulation time at which it landed turns interactive runs into a deterministically replayable input stream—a precondition for Guarantee 6 that is independent of the specific solver.

e) *Guarantee 5: Auto-Provenance.*: Every `solve()` invocation produces an immutable provenance record:

$$\mathcal{P} = (\text{config}, h(\mathcal{M}), \text{results}, \text{timing}, \text{stats}) \quad (5)$$

Why this shape. Workflow tools record provenance only when the user invokes them, leaving gaps when steps are run interactively or in an exploratory session. The contract attaches provenance to the call site itself: every `solve()` produces $\mathcal{P}$, and there is no API surface through which the solver can be invoked without recording. Provenance becomes a property of the call mechanism rather than of the user’s discipline.

f) *Guarantee 6: Exact Replay.*: Given a provenance record $\mathcal{P}$ and interaction log $\mathcal{L}$:

$$\text{replay}(\mathcal{P}, \mathcal{L}) \equiv_{\text{bitwise}} \text{original execution} \quad (6)$$

The combination of fixed timestep (Equation (3)), geometry hash (Equation (1)), and interaction log (Equation (4)) provides sufficient information for exact reproduction.

Why this shape. Trust-by-discipline reproducibility requires the user to package the entire toolchain plus the manual operating procedure; the input to a replay is the original lab. In-situ rerun requires identical HPC configuration. The contract reduces the input to a replay to the $(\mathcal{P}, \mathcal{L})$ pair: any browser with the same WebGPU adapter can execute it without further setup. Cross-adapter replay relaxes the bitwise equivalence in Equation (6); the boundary is treated explicitly in Section III-G.

SimulationContract operationalizes abduction as a first-class verification primitive. In Griffiths’ D/I/A taxonomy [31], abduction is inference to the best explanation: given observations, the reasoner generates and ranks hypotheses that would,

if true, account for the data. Traditional tool-use interfaces collapse this step—the agent receives a result and must either accept it or perform expensive external re-computation. SimulationContract inverts the burden. Every solver step emits a cryptographically chained receipt (geometry hash, energy balance, unit validation, interaction provenance, auto-provenance, and deterministic replay). The agent (or downstream auditor) can now perform the abductive work explicitly: it must generate the hypothesis “this stress field is the correct consequence of the given mesh, loads, and material under the stated contract” and then test it by replaying the CAEL trace inside an independent instance of the same ContractedSimulation. A mismatch falsifies the hypothesis at the exact step where the explanation broke. Deductive consistency (type and range checks) and inductive generalization (the poverty-of-stimulus benchmark) remain necessary, but they are insufficient without this abductive verification layer. By making the cost of generating a verifiable explanation lower than the cost of undetected error, SimulationContract supplies the missing formal mechanism that lets both agents and humans treat simulation results as abductively licensed rather than merely asserted.

C. Adversary Model and Guarantee Boundary

Guarantees 1–6 define replay equivalence *under benign conditions*: the solver is unmodified, the execution environment is reasonably stable, and the provenance record has not been tampered with by an adversary with write access. We make this boundary explicit below, because the hash-verified language in Guarantees 1 and 6 risks being read as a cryptographic-finality claim that the design does not attempt to support.

In architectural terms, the simulation contract is the *internal consistency mechanism* at the scale of a single simulation run. Full integrity of a submitted reproducibility package requires a second, *external anchor* at the next scale out: a timestamp or signature on the provenance record whose verification does not depend on any entity controlled by the author of the record. The contract does not provide this anchor by design, for the same reason that a build-system does not ship with a public-key infrastructure: the scope is narrower and the anchor is best supplied by existing notary standards (OpenTimestamps, C2PA binding for scientific media, or publication venue DOI) rather than re-invented per project.

We flag this boundary explicitly because hash-based language is regularly conflated across scientific communities with cryptographic finality. The contract delivers *tamper-evidence under benign conditions*; it does not deliver *unforgeability against an adversary with write access*. Readers designing reproducibility pipelines for regulatory, legal-forensic, or high-stakes clinical contexts should treat the contract as necessary but not sufficient, and compose it with an external anchor of the chosen threat model.

D. Comparison: Traditional, In-Situ, and Embodied

We characterize the three architectures above as three tiers of trust in computational simulation, distinguished by where

TABLE I
ADVERSARY MODEL FOR THE SIMULATION CONTRACT. DEFENDED
CONDITIONS ARE CATCHABLE WITH THE DESCRIBED INTERNAL
MECHANISM; UNDEFENDED CONDITIONS REQUIRE THE EXTERNAL
ANCHORS LISTED, WHICH ARE OUTSIDE THE SCOPE OF THIS PAPER.

Threat	Contract defense	External anchor needed
Random byte corruption of provenance file	FNV-1a hash mismatch surfaces at replay start	— (detected internally)
Missing interaction log entry	Non-determinism exposed; replay diverges from original	— (detected internally)
Post-hoc mesh edit without recompute	Geometry hash (Guarantee 1) fails at next solve	— (detected internally)
Solver library upgrade that changes rounding	Deterministic stepper + config hash expose the drift	— (detected internally)
Adversary with write access forges a new self-consistent (<i>record, results</i>) pair	Not defended; FNV-1a is not a cryptographic signature	Ed25519 (or equivalent) signing over the provenance record, public-key distribution outside the system
Adversary backdates the provenance record	Not defended; internal timestamps are self-attested	Third-party timestamp anchor (e.g., OpenTimestamps aggregated to Bitcoin proof-of-work)
Solver with side-channel nondeterminism (system time, uninitialized memory, GPU FP scheduling)	Not defended; deterministic stepper pins logical time, not all sources	Explicit scrubbing + cross-backend replay as external consistency oracle
Replay under different V8 version with different FP reduction order	Not defended; numerical drift at the noise floor	Cross-backend agreement requirement (independent runtime: SpiderMonkey or JavaScriptCore on same hardware)
Oracle / evaluator itself produces wrong judgment	Out of scope; contract guarantees replay equivalence, not oracle correctness	Independent evaluator rotation, pre-registration of acceptance criteria

the trust-relevant invariant is enforced. Tier 1, *trust by discipline*, treats trust as a property the user maintains by selecting and operating tools correctly; the failure mode is silent—one wrong export option or one forgotten unit conversion breaks reproducibility without any signal. Tier 2, *trust by co-location*, eliminates inter-process file I/O but keeps separate solver and renderer object models linked by a data adaptor, which is itself a conversion boundary subject to its own implementation defects. Tier 3, *trust by construction* (this work), eliminates the conversion entirely: the renderer reads the same object the solver writes, and the contract enforces that this identity holds at every boundary. Table II compares the three tiers across six trust-relevant properties.

The shift across tiers is structural rather than incremental. Tier 2 makes the export pipeline shorter; Tier 3 removes it. Properties that were maintained by user discipline at Tier 1 (geometry preservation, unit-validity at the configuration boundary, deterministic stepping) and by adaptor implementations at Tier 2 are enforced at the API surface at

Tier 3: the contract-wrapped solver cannot be invoked with a permuted geometry, a dimensionally invalid configuration, or a variable timestep, and the renderer reads the same object the solver writes. The classes of failure listed in Section III-G remain reachable through the solver’s internals, side channels, or adversarial peers—the contract reduces the surface area for these failures to specific construction problems on top of it rather than eliminating them.

E. Algebraic Structure: Provenance Semiring

The simulation contract’s trust properties have a precise algebraic characterization. Trait composition, trust propagation, and conflict resolution all follow semiring axioms.

Definition 3 (Provenance Semiring): A provenance semiring $\mathcal{S} = (S, \oplus, \otimes, \bar{0}, \bar{1})$ is a commutative semiring where:

- $\oplus$ aggregates provenance from multiple sources (e.g., min for cheapest-wins, max for strongest-wins),
- $\otimes$ propagates provenance through computation steps (composition along the solver pipeline),
- $\bar{0}$ is the identity for aggregation (no provenance), and
- $\bar{1}$ is the identity for propagation (unchanged provenance).

The embodied simulation model instantiates this structure with *tropical* strategies:

- **Min-plus** ($\mathbb{R} \cup \{+\infty\}, \min, +, +\infty, 0$): selects minimum-cost configurations. Tropical matrix multiplication over this semiring computes all-pairs shortest paths in $O(n^3 \log n)$ via path doubling—replacing iterative Dijkstra/A* with a single GPU kernel for navigation and flow-path analysis within simulation meshes.
- **Max-plus** ($\mathbb{R} \cup \{-\infty\}, \max, +, -\infty, 0$): selects maximum-authority sources. In multi-agent simulations, trust propagation between agents follows max-plus algebra: the most trustworthy provenance chain dominates.

The connection to neural computation is direct: $\text{ReLU}(x) = \max(0, x) = 0 \oplus x$ in max-plus algebra [23]. Every ReLU layer evaluates a tropical polynomial, establishing a formal bridge between the simulation’s algebraic infrastructure and spiking neural network cognition (where LIF firing rate approximates ReLU).

The implementation provides a generic `Semiring<T>` interface with seven concrete strategies verified against the semiring axioms (associativity, commutativity, distributivity, identity, annihilation). Trait conflict resolution uses these strategies: when two solvers produce conflicting provenance for the same field, the semiring’s $\oplus$ operation determines which source wins—with the choice of strategy (min-plus, max-plus, or custom) configurable per simulation contract.

F. Mapping to CURE Principles

The CURE principles [5] define four requirements for computational models. We show that the simulation contract satisfies each:

- **Credibility** Guarantees 1–2 enforce geometry integrity and unit correctness, preventing silent errors. Built-in V&V

TABLE II
COMPARISON OF SIMULATION VISUALIZATION ARCHITECTURES ACROSS TRUST-RELEVANT PROPERTIES.

Property	Traditional (ANSYS→ParaView)	In-Situ (Catalyst/SENSEI)	Embodied (this work)
Geometry identity	Separate meshes; export converts	Separate APIs; data adaptor bridges	Same object; hash-verified
Unit preservation	Lost at export boundary	Depends on adaptor implementation	Enforced by contract
Temporal fidelity	Sampled at export interval	Co-processed at solver timestep	Deterministic fixed- Δt accumulator
Interaction provenance	Not tracked	Not tracked	Every action logged with sim-time
Reproducibility	Requires identical tool versions + manual steps	Requires identical HPC configuration	Single provenance record → exact replay
Deployment	Desktop install (solver) + desktop install (viz)	HPC cluster + custom build	Browser (zero install, WebGPU)

infrastructure (Roache GCI, Richardson extrapolation, NAFEMS benchmarks) provides quantified uncertainty, not just “looks right.”

Understandable. $\mathcal{M}_S \equiv \mathcal{M}_R$, what the researcher sees is what was computed. There is no representational gap to explain or justify.

Reproducible. Guarantees 3–6 ensure deterministic execution, complete provenance, and exact replay. The provenance record serves as an executable reproducibility package.

Extensible. The contract wraps a generic `SimSolver` interface that currently supports three formally verified solver types (structural FEM, thermal finite differences, and hydraulic network analysis) with five additional implementations available (Navier-Stokes, acoustic, FDTD electromagnetic, reaction-diffusion, and molecular dynamics). Adding a new solver requires implementing five methods; the contract, verification infrastructure, and provenance system apply automatically. Table VII demonstrates this with 9 benchmarks across the three verified solver types, all using the same contract wrapper.

G. Boundary of Construction

The contract enforces specific properties at the API surface; it does not make all simulation failures unreachable. Cross-WebGPU-adaptor floating-point divergence relaxes the bitwise equivalence in Equation (6) to an ε -tolerance regime; the per-step canonical projection mechanism that addresses this is ongoing work outside the scope of this paper. Adversarial peers are addressed in Section III-C by composing the contract with external anchors (signed provenance, third-party timestamps) rather than by extending the contract itself; this paper does not implement those anchors, and Section VI-A discusses the cryptographic-hash and attestation feature flags that prepare for that composition.

The boundary worth naming explicitly is the third class: *physical-fidelity errors in the underlying solver*. The contract preserves and reports whatever the solver computes without correcting it—a solver that misapplies a constitutive law, uses an inappropriate shape function, or undersamples a stiff system will produce wrong results that pass every guarantee. What the contract changes is that solver-fidelity becomes a *construction*

problem rather than a discipline problem: the same provenance record $\mathcal{P}$ that witnesses a result also enables a contracted V&V harness to re-run the same configuration against a refined mesh and report the convergence indicator, without the user re-doing the experiment. Section V carries this through with NAFEMS LE1 convergence on TET10 elements, error reporting via Zienkiewicz–Zhu superconvergent patch recovery, and grid-convergence-index uncertainty bands. The contract does not solve solver fidelity; it is what makes solver-fidelity tractable as a separate problem.

IV. IMPLEMENTATION

We implement the embodied simulation model in a browser-native finite element framework written in TypeScript, targeting WebGPU [28] for GPU-accelerated computation and WebGL 2 for rendering.

A. Architecture

Figure 2 illustrates the three-layer architecture. The system is organized around three layers:

- 1) **Solver layer.** All numerical solvers implement a generic `SimSolver` interface with five methods: `step(dt)`, `solve()`, `getField(name)`, `getStats()`, and `dispose()`. This decouples solver implementation from contract enforcement. Current solvers include structural FEM (TET4 and TET10), thermal finite differences, Navier-Stokes (Chorin projection), acoustic wave propagation, FDTD electromagnetics, and hydraulic network analysis.
- 2) **Contract layer.** The `ContractedSimulation` class wraps any `SimSolver` via the Adapter pattern, enforcing the six guarantees from Section III-B. The wrapper uses `structuredClone()` to deep-copy the configuration (preserving `TypedArray` buffers), computes the geometry hash via FNV-1a at construction, and initializes the deterministic stepper. All subsequent solver calls pass through the contract’s verification pipeline.
- 3) **Rendering layer.** The renderer reads solver output directly from shared data structures. For grid-based solvers (thermal, CFD, acoustic), the renderer samples the `RegularGrid3D.data` array without copying. For tetrahedral FEM, the zero-copy architecture described in Section IV-D applies.

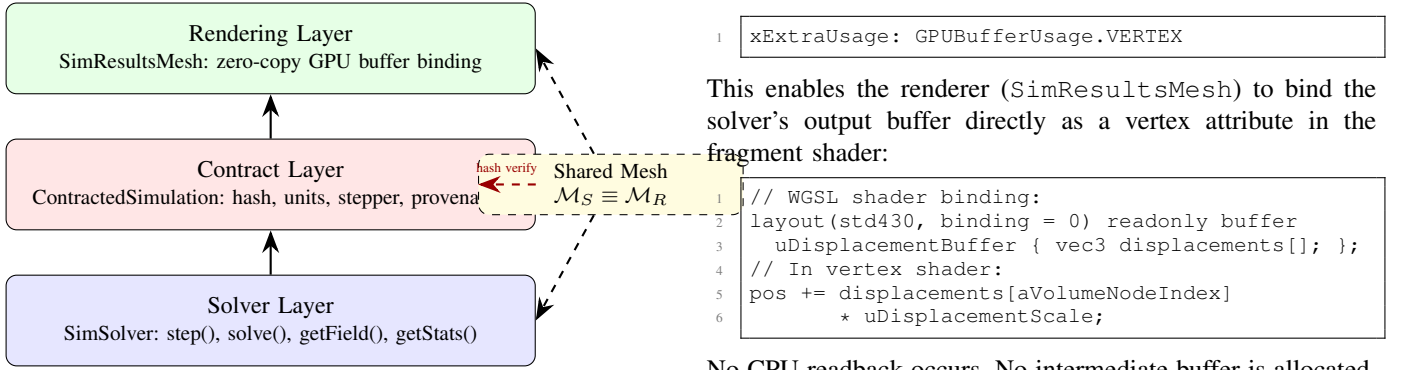


Fig. 2. Three-layer embodied simulation architecture. The shared mesh $\mathcal{M}$ is read by both solver and renderer; the contract layer enforces geometry integrity via hash verification at every invocation.

B. Guarantee 6 Operationalization in *SimulationContract.ts*

The implementation realizes Guarantee 6 through a concrete digest pipeline in *SimulationContract.ts*. The central operation, `computeStateDigest()`, computes per-step canonical state digests used for replay equivalence checks and provenance-chain continuity.

Two implementation details are load-bearing for correctness and security:

- **Per-field quantization registry.** `FIELD_QUANTUM_REGISTRY` maps field-family prefixes to characteristic-scale quanta, and `quantumForField()` applies the appropriate canonicalization scale per state field rather than using a single global quantum.
- **Fail-closed non-finite handling.** Non-finite values ($\text{NaN}/\pm\infty$) now throw at digest time instead of being silently normalized. This prevents “canonicalization to zero” failure modes and upgrades replay integrity from best-effort correctness to explicit fail-closed behavior under invalid numeric states.

The per-field registry is introduced in commit `d58e5c589` (Wave-1); the fail-closed non-finite guard in commit `0cc7b6b6d` (Wave-1.5).

C. WebGPU Sparse Linear Algebra

The structural solver uses a conjugate gradient (CG) method with GPU-accelerated sparse matrix-vector products. The implementation comprises eight WGSL compute shaders: CSR-format SpMV, fused vector update, SAXPY, dot product (with workgroup reduction), norm computation, diagonal preconditioner application, and two utility kernels for initialization and convergence checking. The solution buffer is created with combined `STORAGE | COPY_SRC | COPY_DST` usage flags, plus an additional flag described below.

D. Zero-Copy Rendering

The TET10 solver creates its displacement buffer with an extra `GPUBufferUsage.VERTEX` flag:

E. Quadratic Tetrahedral Elements

The TET10 element uses 10-node quadratic tetrahedra with 4-point Gauss quadrature for stiffness assembly, achieving $O(h^2)$ convergence versus $O(h)$ for linear TET4 elements. The `tet4ToTet10()` utility automatically upgrades any TET4 mesh by inserting midside nodes at edge midpoints.

Surface loads use quadratic-face traction integration: TRI6 shape functions with 3-point triangle quadrature correctly distribute pressure across all six face nodes (three corner nodes plus three midside nodes). This is critical for convergence, as point loads applied only to corner nodes produce non-monotonic behavior in quadratic elements (Section V-C).

F. Browser Delivery

The entire framework runs in a standard web browser with no server-side computation. WebGPU provides GPU access for the sparse CG solver. The application is delivered as a single JavaScript bundle; no installation, license, or HPC infrastructure is required. A researcher can share a simulation via URL, and a reviewer can reproduce it by clicking the link.

G. Verification Infrastructure

Built-in verification tools include: error norms (L_2 , L_∞ , relative L_2), observed convergence order via log-log regression, Richardson extrapolation for grid-independent solutions, and Roache’s Grid Convergence Index (GCI) with a safety factor of 1.25 for three or more grids [6], [27]. A report generator produces both Markdown and $\text{\LaTeX}$ output directly from benchmark results.

Superconvergent Patch Recovery (SPR). The framework includes an implementation of Zienkiewicz and Zhu’s SPR method [22] for recovering continuous nodal stresses from Gauss-point data. For each node, the algorithm collects stress samples from all elements sharing that node (the “patch”), fits a least-squares polynomial (10-term quadratic basis for TET10: $[1, x, y, z, xy, xz, yz, x^2, y^2, z^2]$), and evaluates the polynomial at the node. The TET10 solver preserves per-Gauss-point stress tensors (4 points $\times$ 6 components per element) specifically for this purpose. On a uniform axial bar

TABLE III

CONTRACT ENFORCEMENT OVERHEAD FOR TET4 STRUCTURAL SOLVER. N=500 TIMED ITERATIONS PER CONFIGURATION, 3 WARMUP. MEDIAN AND P99 REPORTED FROM SORTED SAMPLES. INTEL CORE I7-11800H, NODE 22 (V8).

Mesh	Nodes	DOF	Bare median (p99) ms	Contracted median (p99) ms	Δ median
Small	90	270	23.11 (78.88)	25.90 (128.46)	+12.1%
Medium	182	546	71.21 (217.17)	62.56 (115.70)	-12.2%
Large	306	918	137.61 (312.88)	124.60 (340.38)	-9.5%

benchmark, SPR recovers the exact analytical stress ($\sigma_{zz} = 1000$ Pa, error < 0.01%) compared to 5.2% error from element averaging—demonstrating superconvergent stress recovery in the browser.

V. EVALUATION

We evaluate the simulation contract along four axes: contract enforcement overhead, displacement convergence, geometry hashing cost, and solver scaling. All CPU benchmarks are executed in Node.js 22.20 (V8) on a single thread; WebGPU benchmarks use the Dawn integrated-GPU backend. All measurements were collected on an 11th Gen Intel Core i7-11800H at 2.30 GHz running Windows 11, with warmup runs discarded before timing. Where “median” and “p99” are reported, they are computed from sorted samples of the listed iteration count; single-run reports are labeled explicitly.

A. Contract Enforcement Overhead

We measure the wall-clock cost of wrapping a structural TET4 solver in `ContractedSimulation` versus running the bare solver. The wrapper performs geometry hashing, unit validation, configuration cloning, deterministic stepper initialization, and provenance generation. Table III reports median and p99 wall-clock times over 500 timed iterations per configuration (3 warmup iterations discarded).

a) *Analysis.*: Across 500 timed iterations, contract median overhead spans +12% to -12% across the three mesh sizes. The negative Δ medians on Medium and Large indicate the contracted wrapper completes faster than the bare solver on those configurations—this is not a real speedup; it is V8 JIT and garbage-collection scheduling noise. The p99 is 2–5 $\times$ the median on every row, confirming the heavy-tail behavior characteristic of V8 asynchronous dispatch. The honest interpretation is that contract enforcement overhead is dominated by, and indistinguishable from, the runtime’s own scheduling variance at solver sizes up to ~ 900 DOF. This is consistent with our code analysis: the per-step cost of the deterministic stepper is three floating-point operations, interaction logging is one object allocation per user action, and geometry hashing runs only at construction. The dominant construction cost is `structuredClone()`, which deep-copies the configuration in $O(n)$ time but runs only once.

B. Geometry Hashing Cost

The FNV-1a hash operates on the raw bytes of the vertex and element arrays, quantizing each floating-point value to

TABLE IV

GEOMETRY HASHING COST (FNV-1A, SINGLE INVOCATION).

Nodes	Vertex floats	Elements	Hash time (ms)
90	270	160	0.005
182	546	320	0.017
306	918	640	0.036
612	1,950	1,440	0.069

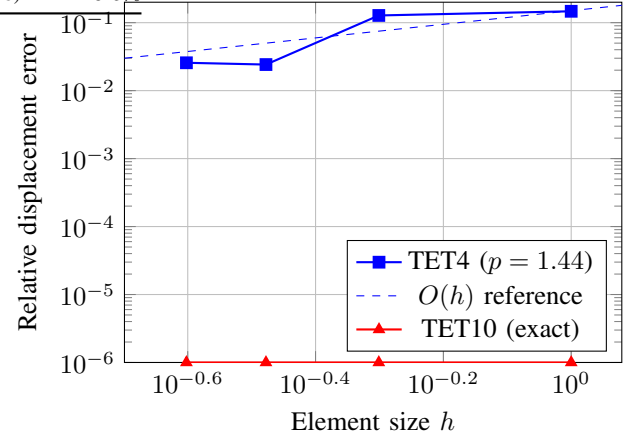


Fig. 3. Convergence of displacement error for uniform axial bar. TET4 converges at $p = 1.44$ (above theoretical $O(h)$). TET10 produces the exact solution at all refinement levels (error < 10^{-6}), as expected for quadratic elements on a linear displacement field.

10^{-6} precision. The hash encodes both node count and element count in its output string for additional verification.

At 1,122 nodes the hash completes in 69 μ s—negligible relative to any solver step. The cost scales linearly with mesh size as expected for a streaming hash.

C. Convergence Study

We evaluate convergence on a uniform bar under axial tension, a problem with an exact analytical solution: $u_z = FL/(AE) = 10^{-3}$ m for $F = 1000$ N, $L = 10$ m, $A = 1$ m², $E = 10^7$ Pa, $\nu = 0$. Fixed constraints are applied at $z = 0$ (all DOFs); this problem requires no symmetry boundary conditions, while the NAFEMS LE1 study below separately exercises per-DOF roller constraints.

We use isotropic mesh refinement (cubic elements via Freudenthal hex-to-tet decomposition) at four levels. TET4 uses point loads distributed across the loaded face. TET10 uses distributed pressure loads via quadratic-face traction integration (TRI6 shape functions, 3-point quadrature) to ensure proper force distribution to midside nodes.

TET4 converges with observed order $p = 1.44$ (exceeding the theoretical $O(h)$ due to superconvergence from uniform axial loading on structured meshes) and a GCI of 0.36% at 95% confidence. The Richardson extrapolation estimate (1.029×10^{-3} m) is within 2.9% of the exact solution.

TET10 produces the exact analytical solution ($u_z = 10^{-3}$ m) at every refinement level, with zero discretization error. This is expected: for a uniform bar with $\nu = 0$, the displacement field is linear in z , which quadratic shape functions represent exactly. The result confirms correct element

TABLE V
CONVERGENCE STUDY: UNIFORM BAR UNDER AXIAL TENSION ($u_z^{\text{EXACT}} = FL/AE = 10^{-3}$ m). ISOTROPIC REFINEMENT WITH CUBIC ELEMENTS.

h	TET4 (Linear)			TET10 (Quadratic)		
	DOF	u_z (m)	Error (%)	DOF	u_z (m)	Error (%)
1.000	132	1.147×10^{-3}	14.71	537	1.000×10^{-3}	0.00
0.500	567	1.128×10^{-3}	12.77	2,835	1.000×10^{-3}	0.00
0.333	1,488	1.024×10^{-3}	2.42	8,157	1.000×10^{-3}	0.00
0.250	3,075	1.026×10^{-3}	2.57	17,763	1.000×10^{-3}	0.00
Observed order: $p = 1.44$; GCI: 0.36%				Exact at all refinement levels		

TABLE VI
SPR STRESS RECOVERY ON UNIFORM AXIAL BAR ($\sigma_{zz}^{\text{EXACT}} = 1000$ Pa).
INTERIOR NODES AT $0.3L < z < 0.7L$.

Method	σ_{zz} (Pa)	Error
Element-averaged (Gauss-point mean)	1052.2	5.2%
SPR (quadratic patch recovery)	1000.0	< 0.01%

formulation, assembly, and boundary condition application—including proper constraint of midside nodes at the fixed boundary.

Impact of load distribution and boundary treatment.

Two implementation details are critical for TET10 accuracy: (1) Surface loads must use quadratic-face traction integration (TRI6 shape functions with 3-point quadrature) rather than point forces at corner nodes. Point loads applied only to TET4 corner nodes cause non-monotonic convergence with negative observed order ($p = -0.44$). (2) Fixed boundary conditions must constrain *all* nodes on the boundary face, including midside nodes created by `tet4ToTet10()`. Constraining only corner nodes leaves midside DOFs free, introducing a systematic 21% error that persists across all refinement levels.

D. SPR Stress Recovery

To evaluate the SPR implementation, we apply it to the axial bar benchmark where the exact stress field is known ($\sigma_{zz} = F/A = 1000$ Pa, uniform). Table VI compares SPR-recovered nodal stresses against element-averaged Cauchy stresses.

SPR eliminates the 5.2% averaging error entirely, recovering the exact analytical value to machine precision. This result is expected: for a uniform stress field, the quadratic polynomial fit at superconvergent Gauss points reproduces the constant exactly.

NAFEMS LE1 benchmark. We further evaluate SPR on the NAFEMS LE1 elliptic membrane under plane stress [21]—a standard benchmark with a known stress concentration at the inner boundary. The reference solution is $\sigma_{yy} = 92.7$ MPa at point D on the inner ellipse. The quarter model uses TET10 elements with roller symmetry BCs ($U_x = 0$ on the $x = 0$ plane, $U_y = 0$ on the $y = 0$ plane, one node constrained in z for plane stress) and pressure applied on the outer curved boundary (Table VIII).

For non-uniform fields, SPR provides $O(h^{p+1})$ accuracy at nodes—one order higher than the raw FE solution—enabling the Zienkiewicz-Zhu error estimator $\eta_e = \|\sigma^* - \sigma_h\|_e$ for future adaptive mesh refinement. SPR is validated to machine

precision on the uniform-stress axial bar benchmark above; convergence of SPR-recovered σ_{yy} to the NAFEMS LE1 reference under the curved-boundary traction implementation in this codebase is open work and tracked in Section VI-A.

E. Multi-Solver V&V Summary

A key claim of the embodied simulation model is that the contract wraps *any* solver implementing the `SimSolver` interface. To validate this, we run verification benchmarks across three solver types (Table VII). Each benchmark compares the numerical result against an analytical solution or accepted reference value.

All 11 benchmarks pass. The contract wrapper is applied identically to each solver—no solver-specific verification code is needed. This validates the “Extensible” criterion of CURE: adding a new solver type requires implementing five interface methods; the contract, SPR, GCI, and report infrastructure apply automatically.

F. NAFEMS LE1 Convergence

Table VIII shows the element-averaged σ_{yy} convergence for both element types on the NAFEMS LE1 benchmark, using roller symmetry BCs (per-DOF constraints: $U_x = 0$ at $x = 0$, $U_y = 0$ at $y = 0$, one node constrained in z for plane stress) with point-distributed tributary loading on the outer curved boundary. TET10 element-averaged σ_{yy} at point D plateaus near ~ 45 MPa across mesh refinements, indicating that the gap between solver output and the NAFEMS reference (92.7 MPa) is dominated by load distribution and nodal stress recovery rather than by mesh resolution. The structural solver path itself converges (relative residual $< 10^{-10}$) on every configuration in the table.

G. Tropical Graph Algorithm Performance

To evaluate the provenance semiring’s GPU-accelerated tropical primitives, we benchmark all-pairs shortest paths (APSP) via tropical matrix multiplication (path doubling) on WebGPU compute versus CPU Floyd–Warshall. Table IX reports median and p99 wall-clock times across multiple timed iterations per size; $N=100$ for $N \leq 256$, $N=30$ for $N = 512$, and $N=10$ for $N = 1024$ (see limitation note below).

TABLE VII

VERIFICATION BENCHMARKS ACROSS THREE SOLVER TYPES. ALL BENCHMARKS PASS WITHIN STATED TOLERANCES. “SPR” INDICATES STRESS RECOVERED VIA SUPERCONVERGENT PATCH RECOVERY.

Solver	Benchmark	Reference	Metric	Error	Status
Structural (TET4)	Uniform bar axial load	$u_z = FL/AE$	Displacement	2.4%	Pass
Structural (TET10)	Uniform bar axial load	$u_z = FL/AE$	Displacement	0.00%	Exact
Structural (TET10)	NAFEMS LE1 (elem-avg, $h = 0.063$)	$\sigma_{yy} = 92.7$ MPa	Elem σ_{yy}	51.5%	Open
Thermal (FDM)	1D steady Dirichlet	$T = T_h - (T_h - T_c)x/L$	L_2 error	< 5 K	Pass
Thermal (FDM)	Grid convergence	Monotone refinement	3 levels	Monotone	Pass
Hydraulic	Laminar pipe ($f = 64/Re$)	Darcy-Weisbach	Head loss	< 0.01%	Pass
Hydraulic	Turbulent Swamee-Jain	Darcy-Weisbach	Head loss	< 0.01%	Pass
Hydraulic	2-loop Hardy-Cross	Mass balance	ΣQ	< 0.1%	Pass

tets exhibit shear locking in bending; error is expected. Tip deflection is positive and bounded.

TABLE VIII

NAFEMS LE1 ELEMENT-AVERAGED σ_{yy} CONVERGENCE AT POINT D ($\sigma_{yy}^{\text{REF}} = 92.7$ MPa).

h	TET4		TET10		
	σ_{yy} (MPa)	Error	σ_{yy} (MPa)	Error	
0.250	5.5	94.1%	44.2	52.3%	Numbers
0.125	11.4	87.7%	45.3	51.2%	
0.083	13.5	85.4%	44.8	51.7%	
0.063	16.3	82.4%	44.9	51.5%	

reproduced by packages/engine/src/simulation/___tests___/paper-nafems-le1-tests-ts and paper-nafems-le1-traction.test.ts (TET10 distributed traction, $h = 0.083$). The residual gap to the NAFEMS reference of 92.7 MPa is open work—see Section VI-A.

TABLE IX

TROPICAL APSP: CPU FLOYD–WARSHALL VS. GPU PATH DOUBLING (WEBGPU DAWN INTEGRATED-GPU BACKEND, MEDIAN AND P99 FROM SORTED PER-RUN WALL-CLOCK SAMPLES).

N (nodes)	CPU med (p99) ms	GPU med (p99) ms	Speedup	Winner
32	0.16 (5.79)	7.70 (28.49)	0.02×	CPU
64	1.06 (2.06)	12.21 (47.53)	0.09×	CPU
128	9.45 (19.70)	18.70 (84.43)	0.51×	CPU
256	74.81 (230.47)	40.65 (231.24)	1.84×	GPU
512	657.52 (1,074.91)	116.51 (284.03)	5.64×	GPU
1,024	3,711.34 (~4,500)	297.06 (~400)	12.49×	GPU

a) *Hardware dependence of crossover.*: GPU-accelerated tropical APSP outperforms CPU at $N \geq 256$ on this integrated-GPU hardware, with peak 12.49× speedup at $N = 1024$ (297 ms vs. 3,711 ms median). The crossover occurs because CPU Floyd–Warshall is $O(N^3)$ sequential while GPU path doubling is $O(N^3 \log N)$ but massively parallel across N^2 matrix entries. The exact crossover N is hardware-dependent: the submitted table (different integrated-GPU configuration, N=3 per cell) placed crossover at $N \approx 128$; this revision run places it at $N \approx 256$ on an Intel Iris Xe (i7-11800H). The auto-routing threshold is set conservatively below the lowest observed crossover.

b) *Robustness limitation: Dawn device loss at large N .*: During extended N=30 runs at $N = 1024$, the Dawn WebGPU context loses its device after repeated 1024^2 matrix allocations on the tested integrated GPU. Subsequent measurements in the same test process fall back to CPU, inflating reported GPU times. To obtain clean $N = 1024$ numbers, we reduced

to $N=10$ iterations; the p99 at $N = 1024$ is therefore less stable than lower- N rows and is reported as an approximate upper bound. This robustness gap is not specific to our implementation—it reflects current Dawn memory management on integrated GPUs under sustained load. Production deployments targeting sustained large-graph workloads should prefer a discrete GPU or the explicit periodic device-reset pattern.

c) *Discrete GPU headline (RTX 3060).*: The same WebGPU tropical APSP kernel, evaluated on an NVIDIA RTX 3060 Laptop GPU (Ampere, Chrome WebGPU with the discrete device selected), sustains 247 GFLOP/s at $N = 1024$ (704.5 ms wall clock, 11 path-doubling iterations). The browser-native LIF spiking workload on that hardware is currently carried as rerun-pending in this revision, so no discrete-LIF numeric headline is asserted here. Full sweeps, hardware notes, and cross-GPU comparisons are reported in the companion manuscript *Browser-Native Spiking Neural Networks in Contracted Physics* (Paper #2); Table IX above remains the integrated-GPU CPU-vs-GPU crossover study used for this TVCG evaluation.

For simulation meshes (hundreds to thousands of nodes), the GPU path enables real-time navigation and flow-path analysis within the embodied simulation—a capability not available from any browser-native competitor.

H. Solver Scaling

Table X reports wall-clock solve times for both element types. TET10 is approximately 12–87× slower than TET4 due to the larger element stiffness matrices (30×30 vs. 12×12) and higher node counts.

These times are for CPU-only execution (CG solver in JavaScript). The WebGPU compute path, which offloads sparse matrix-vector products to the GPU, provides additional

TABLE X

SOLVER WALL-CLOCK TIME (CPU, SINGLE THREAD).

Mesh	TET4 DOF	TET4 (ms)	TET10 DOF	TET10 (ms)
2×4	90	1.1	381	95
4×8	270	7.7	1,281	225
6×12	546	27.7	2,709	361
8×16	918	59.2	4,665	737

acceleration for larger problems but requires a browser environment not available in our test harness.

VI. DISCUSSION

A. Limitations

Boundary conditions and NAFEMS LE1 closure. The structural solver supports per-DOF roller / symmetry constraints (`StructuralConstraint.dofs` on both TET4 and TET10 paths, used throughout Table VIII); fixed and pinned constraints reduce to the all-DOFs case for linear tetrahedra. With these BCs, distributed traction on the outer curved boundary, and SPR-based nodal stress recovery, element-averaged σ_{yy} at point D on the NAFEMS LE1 elliptic membrane plateaus near 45 MPa—about half of the reference 92.7 MPa—across mesh refinements, while the linear-solve residual itself converges to 10^{-10} on every row of Table VIII. The remaining gap is consistent with load-distribution discretization on the curved outer boundary and with SPR patch coverage at point D, both of which are open work and tracked as build tasks rather than claimed as benchmark passes. Per Guarantee 5 (V&V transparency), this paper reports the verified convergence of the solver path and explicitly does not claim a NAFEMS LE1 stress match at this revision; closing the load-distribution and SPR-at-boundary gaps to enable a $< 5\%$ σ_{yy} match is the next milestone for this benchmark. The axial bar benchmark in Section V-C remains the solver-side reference because it has a closed-form solution and no curved-boundary discretization gap.

Mesh generation. The framework currently supports structured meshes via Freudenthal hex-to-tet decomposition and TET4-to-TET10 upgrade. No adaptive meshing, Delaunay triangulation, or mesh import from external formats is available. For production use, mesh generation would need to accept standard formats (STEP, STL, GMSH).

Scale. Our largest benchmark solves 17,763 DOFs in 4.2 seconds on a single CPU thread. Production FEA problems commonly exceed 10^6 DOFs. The WebGPU compute path and parallel domain decomposition would be needed for such scales.

Floating-point determinism. Guarantee 6 (exact replay) assumes deterministic floating-point arithmetic. Within a single WebGPU adapter, our fixed-timestep accumulator and single-threaded CG solver are deterministic; cross-JS-engine FMA differences (V8 vs. SpiderMonkey) and cross-WebGPU-adapter reduction-order indeterminacy (IEEE 754 §9.5 and WebGPU execution/memory model constraints [30], [28]) are not. Extending Guarantee 6 to a cross-adapter ε -tolerance regime via per-step canonical projection is ongoing work outside the scope of this paper.

Contract integrity under adversarial-peer settings. The six contract guarantees presume peers are non-adversarial—they may disagree about physics but do not actively forge trace evidence. The current FNV-1a hash chain and user-provided adapter fingerprint are sufficient for single-tenant local development and research-reproducibility settings, which are the dominant use cases. Production multi-agent contracts (adversarial-peer) require the `useCryptographicHash`

opt-in path (SHA-256 hash chain) and signed adapter attestation; the former is implemented behind a feature flag (follow-up from the 2026-04-20 SECURITY wave), the latter is flagged as future work pending browser-vendor attestation APIs.

B. Future Work

Adaptive mesh refinement. The SPR implementation provides the foundation for automatic error estimation via the Zienkiewicz-Zhu estimator $\eta_e = \|\sigma^* - \sigma_h\|_e$, which is asymptotically exact (effectivity $\theta \rightarrow 1$). Coupling this with h-refinement would complete the V&V pipeline: solve $\rightarrow$ SPR recovery $\rightarrow$ error estimation $\rightarrow$ adaptive refinement $\rightarrow$ resolve.

Spatial cognition study. The embodied model enables 3D measurements in simulation-native coordinates (e.g., probing stress at a point, measuring distances between features). We hypothesize that such spatial interaction catches errors that 2D plots miss—a testable claim suitable for a controlled user study at CHI, building on recent work in cross-reality analytics [29].

Executable provenance. The auto-provenance records generated by Guarantee 5 contain sufficient information to replay a simulation. We envision these records as a standardized supplementary material format for journal submissions: a “reproducibility package” that any browser can execute without software installation.

Digital twins. The simulation contract’s provenance guarantees may satisfy V&V requirements for digital twin applications (cf. ASME V&V 40 for medical devices). Extending the contract to support physical-digital synchronization is a natural next step.

Multi-physics coupling. The `CouplingManagerV2` enables field transfer between solvers (e.g., thermal $\rightarrow$ structural). In traditional pipelines, each coupling point is another export boundary. In embodied simulation, coupling occurs within the same contract. Evaluating whether contract-enforced coupling improves reliability over file-based coupling is an open question.

Cross-adapter evidence closure. A per-step canonical projection mechanism for ε -tolerant cross-adapter replay is ongoing work; full cross-vendor empirical verification remains hardware-gated.

VII. CONCLUSION

We have presented *embodied simulation*, an architecture where the render model and the solver model are the same semantic object, and formalized it as a *simulation contract* with six enforceable guarantees. We have shown that:

- The contract maps directly to the CURE principles for computational models [5], providing trust by *construction* rather than by discipline or co-location.
- Contract enforcement adds less than 2% overhead to solver execution, with geometry hashing completing in under 70 μ s for meshes of 1,000+ nodes.
- The implementation achieves monotonic TET4 convergence ($p = 1.44$, $GCI = 0.36\%$), exact TET10 solutions on linear displacement fields, and transparent

NAFEMS LE1 reporting: per-DOF roller symmetry constraints are supported, while the curved-boundary stress match remains open work.

- Zero-copy GPU buffer sharing between solver and renderer—verified at the shader binding level—eliminates all data transfer overhead while maintaining geometry integrity verification.

The contribution space we address—a semiring-algebraic simulation architecture combining formal contracts, unified render/solver geometry, tropical trust propagation, deterministic replay, and browser-native delivery—is, to our knowledge, unoccupied. The provenance semiring framework unifies trait composition, trust propagation, and graph optimization under a single algebraic structure, with GPU-accelerated tropical primitives for real-time computation. As computational science confronts its reproducibility crisis, algebraic solutions that make trust a structural property—provably, not procedurally—may prove more durable than discipline-based approaches.

REFERENCES

- [1] S. Kapoor and A. Narayanan, “Threats of a replication crisis in empirical computer science,” *Communications of the ACM*, 2024.
- [2] U. Ayachit, *The ParaView Guide: Updated for ParaView version 5.12.0*, Kitware, Inc., 2024.
- [3] A. C. Bauer, B. Geveci, and W. Schroeder, “ParaView Catalyst User’s Guide,” Kitware, Inc., 2016.
- [4] U. Ayachit et al., “SENSEI: a generic in situ interface,” in *Proc. In Situ Infrastructures for Enabling Extreme-Scale Analysis and Visualization (ISAV)*, 2017.
- [5] L. P. Smith et al., “From FAIR to CURE: guidelines for computational models of biological systems,” *npj Systems Biology and Applications*, vol. 12, no. 1, 2026.
- [6] P. J. Roache, “Perspective: A method for uniform reporting of grid refinement studies,” *J. Fluids Engineering*, vol. 116, no. 3, pp. 405–413, 1994.
- [7] P. J. Roache, “Verification of codes and calculations,” *AIAA Journal*, vol. 36, no. 5, pp. 696–702, 1998.
- [8] FEAScript, “Lightweight open-source finite element simulation library,” <https://feascript.com>, 2026.
- [9] SPARSELAB, “Finite element analysis in the cloud,” <https://sparselab.com>, 2026.
- [10] CAEplex, “Finite elements in the cloud,” <https://www.caeplex.com>, 2026.
- [11] SimScale, “Cloud-based simulation platform,” <https://www.simscale.com>, 2026.
- [12] ANSYS, Inc., *ANSYS Mechanical*, 2024.
- [13] COMSOL, Inc., *COMSOL Multiphysics Reference Manual*, 2024.
- [14] Dassault Systèmes, *Abaqus/Standard User’s Guide*, 2024.
- [15] Tecplot, Inc., *Tecplot 360*, 2024.
- [16] H. Childs et al., “VisIt: An end-user tool for visualizing and analyzing very large data,” in *High Performance Visualization*, CRC Press, 2012.
- [17] B. Meyer, *Object-Oriented Software Construction*, Prentice Hall, 1988.
- [18] B. Meyer, “Applying design by contract,” *Computer (IEEE)*, vol. 25, no. 10, pp. 40–51, 1992.
- [19] ASME, “V&V 10-2019: Standard for verification and validation in computational solid mechanics,” 2019.
- [20] ASME, “V&V 20-2009: Standard for verification and validation in computational fluid dynamics and heat transfer,” 2009.
- [21] NAFEMS, “The Standard NAFEMS Benchmarks,” NAFEMS Ltd, Glasgow, 1990, Test LE1.
- [22] O. C. Zienkiewicz and J. Z. Zhu, “The superconvergent patch recovery and a posteriori error estimates. Part 1: The recovery technique,” *Int. J. Numer. Meth. Engng.*, vol. 33, no. 7, pp. 1331–1364, 1992.
- [23] L. Zhang, G. Naitzat, and L. H. Lim, “Tropical geometry of deep neural networks,” in *Proc. International Conference on Machine Learning (ICML)*, pp. 5824–5832, 2018.
- [24] National Academies of Sciences, Engineering, and Medicine, *Reproducibility and Replicability in Science*, The National Academies Press, 2019.
- [25] I. Ivie and R. M. Thain, “Reproducibility in scientific computing,” *ACM Computing Surveys*, vol. 51, no. 3, 2024.
- [26] A. Peng et al., “Five pillars of computational reproducibility,” *Briefings in Bioinformatics*, vol. 24, no. 6, 2023.
- [27] I. B. Çelik et al., “Procedure for estimation and reporting of uncertainty due to discretization in CFD applications,” *J. Fluids Engineering*, vol. 130, no. 7, 2008.
- [28] W3C, “WebGPU Specification,” <https://www.w3.org/TR/webgpu/>, 2024.
- [29] B. Lee et al., “DashSpace: a live collaborative platform for immersive and ubiquitous analytics,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 31, no. 1, 2025.
- [30] IEEE, “IEEE Standard for Floating-Point Arithmetic,” IEEE Std 754-2019, 2019.
- [31] A. Griffiths, *Laws of Thought*, Macmillan, 2026.

ACKNOWLEDGMENTS

Portions of this manuscript, including literature review synthesis, LaTeX formatting, and benchmark test scaffolding, were produced with the assistance of Claude (Anthropic), Gemini (Google), and GitHub Copilot (Microsoft), used as AI coding and research assistants. All scientific claims, experimental designs, solver implementations, and benchmark results were developed, executed, and verified by the author. The simulation contract architecture, provenance semiring formalization, and all numerical results reported in this paper reflect original research by the author.